

Q.1

Q1.a — Apply Master Theorem to the recurrences (6 marks)

Master theorem for recurrence $T(n) = aT(n/b) + f(n)$

- Let $n \log_b a$ be the critical function.
- Compare $f(n)$ with $n \log_b a$.

i) $T(n) = 4T(n/2) + n$

- Here $a=4, b=2 \Rightarrow n \log_b a = n \log_2 4 = 2n$. $a=4, b=2 \Rightarrow n^{\log_b a} = n^2$.
- $f(n) = n = O(n^{2-\epsilon})$ for $\epsilon=1$. So this is **Case 1** of Master theorem.
- Therefore $T(n) = \Theta(n \log_b a) = \Theta(n^2)$.

ii) $T(n) = 4T(n/2) + n^2$

- $a=4, b=2 \Rightarrow n \log_b a = n^2$.
- $f(n) = n^2 = \Theta(n \log_b a)$, so **Case 2** applies.
- Therefore $T(n) = \Theta(n^2 \log n)$.

Q1.b — Define Θ and prove

$4n^2 + 3n \in \Theta(n^2)$ (4 marks)

Definition: $f(n) \in \Theta(g(n))$ if there exist positive constants c_1, c_2, n_0 such that for all $n \geq n_0$, $c_1 g(n) \leq f(n) \leq c_2 g(n)$.

Take $f(n) = 4n^2 + 3n$, $g(n) = n^2$. For $n \geq 1$,

- Lower bound: $4n^2 + 3n \geq 4n^2$. So choose $c_1 = 4$.
 - Upper bound: $4n^2 + 3n \leq 7n^2$ for $n \geq 1$. Choose $c_2 = 7$.
- Thus for $n_0 = 1$, $4n^2 \leq 4n^2 + 3n \leq 7n^2$, so $4n^2 + 3n \in \Theta(n^2)$.

Q1.c — Define best/avg/worst case and analyze given code (4 marks)

Definitions:

- **Best case:** minimum time over inputs of size n .
- **Average case:** expected time over a reasonable probability distribution of inputs.
- **Worst case:** maximum time over inputs of size n .

Given code:

```
void sort(int a[], int n) {
    int i, j, temp;
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j + 1]) {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
}
```

This is **Bubble Sort** (inner loop compares adjacent elements and swaps).

Time complexities:

- **Best case:** if array already sorted and implementation *does not* have early-exit, complexity is still $O(n^2)$. If implementation includes an optimized early-exit (detects no swaps) then best-case is $O(n)$.
- **Average case:** $\Theta(n^2)$.
- **Worst case:** $\Theta(n^2)$ (reverse-sorted input).

(Justification: two nested loops—outer up to n , inner up to $\approx n$ — $O(n^2)$.)

Q.2

Q2.a — Control abstraction for divide & conquer; QuickSort recurrence and solve (6 marks)

Control abstraction (Divide & Conquer):

1. **Divide:** split input of size n into smaller subproblems (often of size n/b).
2. **Conquer:** solve each subproblem recursively.
3. **Combine:** merge subproblem solutions to form final solution.

General recurrence: $T(n) = \sum_{i=1}^k T(n_i) + g(n)$ where $g(n)$ is cost to divide/combine.

QuickSort recurrences:

- **Best/Average case:** ideal split into two roughly equal parts:

$$T(n) = 2T(n/2) + \Theta(n). \quad T(n) = 2T(n/2) + \Theta(n). \quad T(n) = 2T(n/2) + \Theta(n).$$

By Master theorem (with $a=2, b=2, c=2, d=2, n \log_b a = n \log_2 2 = n$, and $f(n) = \Theta(n)$), we get

$$T(n) = \Theta(n \log n). \quad T(n) = \Theta(n \log n). \quad T(n) = \Theta(n \log n).$$

- **Worst case:** pivot always partitions poorly (one part size $n-1$):

$$T(n) = T(n-1) + \Theta(n). \quad T(n) = T(n-1) + \Theta(n). \quad T(n) = T(n-1) + \Theta(n).$$

Solve by substitution/unrolling:

$$\begin{aligned} T(n) &= T(n-1) + cn = T(n-2) + c(n-1) + cn = \dots = T(1) + c \sum_{k=2}^n k = \Theta(n^2). \\ T(n) &= T(n-1) + cn = T(n-2) + c(n-1) + cn = \dots = T(1) + c \sum_{k=2}^n k = \Theta(n^2). \end{aligned}$$

So worst-case $T(n) = \Theta(n^2)$.

Q2.b — Choose QuickSort or MergeSort for given datasets (4 marks)

We assume standard QuickSort (in-place) and MergeSort (stable, $O(n \log n)$ worst-case).

- **Set-1:** already sorted ascending.
 - o If QuickSort chooses a bad pivot deterministically (e.g., first element), this yields worst-case $O(n^2)$. MergeSort remains $O(n \log n)$. So **MergeSort** is safer for already-sorted input unless QuickSort uses a randomized/median pivot. If QuickSort uses randomized pivot or median-of-three, QuickSort is fine and often faster in practice.
- **Set-2:** reverse sorted.
 - o Similar to Set-1: deterministic QuickSort with first-element pivot $\rightarrow$ worst-case $O(n^2)$. MergeSort $\rightarrow O(n \log n)$. So **MergeSort** preferred if pivot selection not robust.
- **Set-3:** mixed (roughly random).
 - o For random-like inputs, **QuickSort** (with good pivot selection) is typically **preferable** due to better in-place behavior and smaller constant factors; complexity $O(n \log n)$ average. MergeSort is also $O(n \log n)$ and stable; choose QuickSort for in-place and practical speed unless stability is required.

Q2.c — Algorithm for iii-th selection problem (QuickSelect) (4 marks)

QuickSelect (expected linear time):

- Choose a pivot (randomized or deterministic).
- Partition array around pivot into LLL (less), EEE (equal), GGG (greater).
- If $|L| \geq i$ or $|L| \geq i$, recurse on LLL.
- Else if $|L| + |E| \geq i$ or $|L| + |E| \geq i$, pivot is answer.
- Else recurse on GGG with adjusted index $i - |L| - |E|$.

Time complexity: expected $O(n)$; worst-case $O(n^2)$ for bad pivot choices (can be improved to deterministic linear by median-of-medians pivot selection).

Q.3

Q3.a — 0/1 Knapsack by DP (tabulation) (6 marks)

Given:

- Weights $w = [2, 1, 5, 3]$ (items 1..4),
- Values $v = [300, 200, 400, 500]$,
- Capacity $W = 5$.

DP formula (tabulation):

Let $DP[i][w]$ = max value using first i items with capacity w .

$$DP[i][w] = \begin{cases} DP[i-1][w], & w < w_i \\ \max(DP[i-1][w], DP[i-1][w-w_i] + v_i), & w \geq w_i \end{cases}$$

Build table for $i = 0 \dots 4$, $w = 0 \dots 5$.

Filled table $DP[i][w]$ (rows i , columns capacity w):

$i \backslash w$:	0	1	2	3	4	5
$i=0$	[0, 0, 0, 0, 0, 0]					
$i=1$	[0, 0, 300, 300, 300, 300]	(item1 $w=2, v=300$)				
$i=2$	[0, 200, 300, 500, 500, 500]	(item2 $w=1, v=200$)				
$i=3$	[0, 200, 300, 500, 500, 500]	(item3 $w=5, v=400$)				
$i=4$	[0, 200, 300, 500, 700, 800]	(item4 $w=3, v=500$)				

- **Maximum value** = $DP[4][5] = 800$.

Items selected (backtracking):

- From table, selected items are **item1 (w=2,v=300)** and **item4 (w=3,v=500)**. Total weight $2+3=5$, total value $300+500=800$.
-

Q3.b — Compare approaches for n th Fibonacci (4 marks)

Approaches:

- Naïve recursion:** $F(n)=F(n-1)+F(n-2)$ (exponential time $O(\varphi^n)$). Not practical.
- Top-down DP (memoization):** stores computed values. Time $O(n)$, space $O(n)$.
- Bottom-up DP (iterative):** compute $F(0)\dots F(n)$ iteratively. Time $O(n)$, space $O(1)$ if only last two values kept.
- Matrix exponentiation / fast doubling:** uses exponentiation by squaring giving $O(\log n)$ time (with large-integer costs). Good for very large n .
- Closed form (Binet's formula):** uses floating-point and roots of quadratic — not exact for large n due to precision.

Best in practice: For usual ranges, **iterative DP ($O(n)$ time, $O(1)$ space)** is simplest and fastest. For extremely large n where asymptotic matters, **fast-doubling / matrix exponentiation ($O(\log n)$)** is best.

Q3.c — TSP dynamic programming recurrence and complexity (4 marks)

Held–Karp (DP) recurrence: Let $T[S,v]$ be the minimum cost to start at source (say vertex 1), visit all vertices in subset S (which includes 1 and v), and end at v . For subset S and $v \in S$, $v \neq 1$:

$$T[S,v] = \min_{u \in S, u \neq v} (T[S \setminus \{v\}, u] + \text{dist}(u, v)).$$

$$\text{Base: } T[\{1\}, 1] = 0.$$

Time/space complexity: There are 2^{n-1} subsets and $O(n)$ choices of end vertex v per subset; computing each entry requires $O(n)$ work — so **time $O(n^2 2^n)$** (some formulations give $O(n^2 2^n)$ or $O(n 2^n)$), and **space $O(n 2^n)$** . This is exponential and only practical for small n .

Q.4

Q4.a — Huffman tree and codes; check prefix-free (6 marks)

Frequencies:

- h:1, e:1, l:3, o:2, w:1, r:1, d:1

Build Huffman tree (summary): combine two smallest repeatedly.

One optimal set of codes produced (one valid Huffman solution is):

```
l : 10
o : 01
h : 000
r : 001
w : 110
d : 1110
e : 1111
```

(There may be other valid Huffman codes with same code lengths depending on tie choices.)

Is the code prefix-free? Yes. Huffman codes are prefix-free by construction (no codeword is a prefix of another) because every symbol is assigned to a distinct leaf in a binary tree.

(If needed, you can show the small tree where each leaf corresponds to one letter — each leaf path is a code; internal nodes correspond to combined frequencies.)

Q4.b — Kruskal better for sparse graphs & Prim for dense graphs — true? Justify. (4 marks)

Short answer: The statement is *generally* true because of algorithmic cost characteristics.

Reasoning:

- **Kruskal's algorithm:** sorts all edges $O(E \log E)$ and repeatedly performs Union-Find operations; total time typically $O(E \log E)$ (or $O(E \alpha(V))$ with counting sort or special structures). For **sparse graphs** where E is $O(V)$, sorting edges is cheap and Kruskal is efficient.
- **Prim's algorithm:** when implemented with an adjacency matrix and simple selection, runs in $O(V^2)$ which is good when E is large (dense graphs where $E \approx V^2$). With a binary heap and adjacency lists Prim runs in $O(E + V \log V)$; for dense graphs E dominates and Prim tends to perform well.

Thus **Kruskal** is typically preferred for sparse graphs (fewer edges), and **Prim** (with suitable implementation) is often preferred for dense graphs. The statement is **true** when interpreted in terms of practical runtime constants and common implementations.

Q.5

Q5.a — Job sequencing with deadlines and profits (Greedy) (6 marks)

Jobs (name, profit, deadline):

```
j1: profit 35, d=3
j2: profit 30, d=4
j3: profit 25, d=4
j4: profit 20, d=2
j5: profit 15, d=3
j6: profit 12, d=1
j7: profit 5, d=2
```

Greedy algorithm: sort jobs by profit descending and assign each job to the latest available slot $\leq$ its deadline.

Sorted by profit: j1(35,d3), j2(30,d4), j3(25,d4), j4(20,d2), j5(15,d3), j6(12,d1), j7(5,d2).

Let maximum number of slots = max deadline = 4. Fill slots 1..4:

- Place **j1** into latest free slot $\leq 3 \rightarrow$ slot 3.
- Place **j2** into latest free slot $\leq 4 \rightarrow$ slot 4.
- Place **j3** into latest free slot $\leq 4 \rightarrow$ slot 2 (slot 4 occupied, slot 3 occupied, next free ≤ 4 is slot 2).
- Place **j4** wants slot ≤ 2 but slot 2 now occupied — check slot 1 $\rightarrow$ empty? Actually earlier j3 took slot 2; slot 1 free — place j4 at slot 1? Wait follow proper greedy: fill latest available $\leq$ deadline:
 - o After j1 (slot 3), j2 (slot 4), for j3: latest available ≤ 4 is slot 2 (slot 4 & 3 used) so place j3 at slot 2.
 - o Remaining slots: none ≤ 2 except slot 1.
- Next jobs cannot improve since all slots 1..4 are now filled (slots: 1=j4? Need final mapping below).

Final scheduled slots (1..4):

slot1 = j4 (profit 20) — determined when j4 was placed (it finds slot 1)

slot2 = j3 (profit 25)

slot3 = j1 (profit 35)

slot4 = j2 (profit 30)

Total profit = 20 + 25 + 35 + 30 = 110.

So the selected job sequence (by slots) is: $[j_4, j_3, j_1, j_2]$ with maximum profit **110**.

(This is the standard greedy solution; different tie handling yields same or equal profit.)

Q5.b — Explain P and NP classes and relationship (4 marks)

P (Polynomial-time): set of decision problems solvable by a deterministic Turing machine in polynomial time (time $O(n^k)$ for some constant k). These are considered efficiently solvable.

NP (Nondeterministic Polynomial-time): set of decision problems for which, if the answer is “yes,” there exists a certificate (witness) verifiable in polynomial time by a deterministic Turing machine. Equivalently solvable in polynomial time by a nondeterministic Turing machine.

Relationship:

- Clearly $P \subseteq NP$ (if a problem is solvable in polynomial time, then a solution can certainly be verified in polynomial time).
 - Whether $P = NP$ is an open problem (the central question of computational complexity).
 - NP-Complete problems are the hardest problems in NP: if any NP-complete problem is in P, then $P = NP$.
-

Q.6

Q6.a — Prove that $O(n \log n)$ is polynomial time (6 marks)

Polynomial-time algorithms are those whose running time is $O(n^k)$ for some constant k .

We show $n \log n$ is bounded above by a polynomial n^2 (for sufficiently large n):

For all $n \geq 2$, $\log_2 n \leq n$. So

$$n \log_2 n \leq n \cdot n = n^2.$$

Hence $n \log n = O(n^2)$. Since n^2 is a polynomial, $n \log n$ is polynomial in n . Therefore any algorithm running in $O(n \log n)$ runs in polynomial time (belongs to class P).

(You can also show $\forall \epsilon > 0, n \log n = o(n^{1+\epsilon})$ for all $\epsilon > 0$; $n \log n = o(n^{1+\epsilon}) \forall \epsilon > 0$, $n \log n = o(n^{1+\epsilon})$, so it is polynomial.)

Q6.b — Key difference between Monte Carlo and Las Vegas randomized algorithms (4 marks)

Monte Carlo algorithms:

- Always run in bounded (usually deterministic) time.
- May produce an incorrect result with some (usually small) probability.
- Example: randomized primality tests that can falsely say "prime" with small probability.

Las Vegas algorithms:

- Always produce a correct result (zero-error probability).
- Running time is a random variable; they may take longer on some runs but expected running time is bounded.
- Example: randomized QuickSort (with random pivot) — always sorts correctly; running time expected $O(n \log n)$.

Summary: Monte Carlo trades correctness for time (bounded time, small error probability); Las Vegas trades determinism of time for guaranteed correctness (correct always, random running time).